Thank you for watching this N-Educate tutorial.

In this video, you will learn how to prepare a completed Google AI Studio Android app so GitHub Actions can automatically build the debug APK and publish it under GitHub Releases for direct downloading on your phone.

This method requires the project to use **Android Gradle Plugin 9.1.1**. It also creates a temporary **debug keystore** during the GitHub Actions build when one is not already present, so the APK can be generated successfully without adding or exposing a permanent keystore.

For further assistance, support and access to more prompts, codes and step-by-step resources, join our Skool community using the link in the video description.

Google AI Studio Prompt to Build and Publish the Debug APK

Prompt Start Here:

—----------

Prepare this completed Google AI Studio Android app for automatic debug APK building and GitHub Release publishing with GitHub Actions.

Before creating anything, confirm that:

gradle/libs.versions.toml

contains exactly:

agp = "9.1.1"

If the AGP version is not exactly 9.1.1:

- Stop immediately.
- Report the detected AGP value.

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.

NO THANKS USE THE APP

.github/workflows/build-apk.yml

Do not create, edit, delete, rename, format, upgrade, or replace any other repository file.

Do not modify the Android app, source code, design, features, package name, application ID, manifest, resources, dependencies, Gradle project files, signing configuration, or project settings.

The workflow must build the debug APK, preserve it as a normal GitHub Actions artifact, and publish the same APK directly under GitHub Releases for easy downloading on a phone.

Workflow triggers

Create a GitHub Actions workflow that:

- Runs automatically on every push to the main branch.
- Also supports workflow_dispatch.
- Does not run on pull requests.
- Uses ubuntu-latest.
- Checks out the complete repository.
- Runs all Gradle commands from the repository root.

Workflow permissions

Add only the permission required to create a GitHub Release:

permissions:
contents: write

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.

NO THANKS USE THE APP

${{ github.token }}

Java setup

Set up Temurin Java 17 using actions/setup-java.

Java must be configured before Gradle is configured or any Gradle command is executed.

Add a step named:

Verify Java environment

That step must:

- Run java -version.
- Print JAVA_HOME.
- Detect the active Java major version.
- Confirm that the active Java major version is 17 or newer.
- Fail clearly if Java is lower than 17.
- Fail clearly if the Java version cannot be detected.

Gradle setup

Set up exactly Gradle 9.3.1 using gradle/actions/setup-gradle.

Do not use:

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.
NO THANKS USE THE APP

- current
- release-candidate
- nightly
- Any Gradle version other than exactly 9.3.1

Run all Gradle commands from the repository root.

Temporary debug keystore

Before building, add a step named:

Create temporary debug keystore

That step must:

- Check whether ./debug.keystore already exists.
- Generate it only when it is missing.
- Use keytool from the configured Java 17 installation.
- Use exactly:
  - Keystore password: android
  - Key alias: androiddebugkey
  - Key password: android

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.
NO THANKS USE THE APP

    - Validity: 10000 days

- Confirm that ./debug.keystore exists after generation.
- Fail clearly if the keystore cannot be created.
- Never commit the keystore.
- Never cache the keystore.
- Never print the keystore.
- Never expose the keystore.
- Never preserve the keystore after the workflow runner is removed.
- Never upload the keystore as an artifact or release asset.

Build the debug APK

Build the debug APK using exactly:

gradle :app:assembleDebug --stacktrace --no-daemon

Do not replace this command with:

- ./gradlew
- gradlew
- assembleRelease

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.
NO THANKS USE THE APP

- Any other Gradle command

Verify the APK

After the build, verify that this exact file exists:

app/build/outputs/apk/debug/app-debug.apk

If the exact file is missing:

- Search only inside app/build/outputs/apk.
- Print the APK paths discovered inside that directory.
- Do not search the entire repository.
- Do not select a release APK.
- Do not select an Android test APK.
- Do not select an APK from another module.
- Do not upload or release an unrelated APK.
- Fail clearly if the expected debug APK does not exist.

Confirm that the selected APK:

- Exists.
- Is a regular file.

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.

NO THANKS USE THE APP

Upload the GitHub Actions artifact

After APK verification, add a step named:

Upload debug APK artifact

Use actions/upload-artifact.

Upload only:

app/build/outputs/apk/debug/app-debug.apk

Name the artifact exactly:

app-debug-apk

Configure artifact upload to fail when the APK is missing.

Do not upload a directory containing unrelated files.

Do not upload:

- debug.keystore
- Credentials
- Source code
- Gradle caches
- Build caches

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.
NO THANKS USE THE APP

- google-services.json
- Firebase configuration files
- Signing files

The normal GitHub Actions artifact must remain available even after GitHub Release publishing is added.

Prepare the APK release asset

After the Actions artifact has been uploaded successfully, add a step named:

Prepare APK release asset

That step must:

- Create a temporary directory inside the GitHub Actions runner for the release asset.
- Do not create or commit this directory in the repository.
- Copy only the verified APK into that temporary directory.
- Give the copied APK a clear filename based on the repository name and workflow run number.
- Sanitize the repository name before using it in the filename.
- Use only safe filename characters.
- Preserve the .apk extension.
- Do not compress the APK.

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.
NO THANKS USE THE APP

- Export the final release-asset path through GITHUB_OUTPUT for the release step.

Use a release filename in this general format:

repository-name-debug-build-run-number.apk

For example:

my-android-app-debug-build-12.apk

Do not hard-code a specific app name or repository name.

Publish the APK to GitHub Releases

After the normal Actions artifact has been uploaded and the release asset has been prepared, add a step named:

Publish APK to GitHub Releases

Use the GitHub CLI command:

gh release create

Do not use an unverified third-party GitHub Release action.

Authenticate GitHub CLI using:

GH_TOKEN: ${{ github.token }}

Before attempting release creation:

- Run gh --version.

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.
NO THANKS USE THE APP

- Do not attempt release creation if APK validation fails.

Unique release tag

Create a unique tag for every workflow execution using both:

- GITHUB_RUN_NUMBER
- GITHUB_RUN_ATTEMPT

Use this tag format:

debug-apk-build-GITHUB_RUN_NUMBER-GITHUB_RUN_ATTEMPT

For example:

debug-apk-build-12-1

Construct the tag dynamically at workflow runtime.

Do not use a fixed tag such as:

- latest
- debug
- build
- v1
- apk

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.
NO THANKS USE THE APP

- The workflow is run manually.
- A failed workflow is rerun.
- The same workflow run is attempted again.

Release contents

Create a published GitHub Release with:

Release title:

Debug APK Build #GITHUB_RUN_NUMBER

Release tag:

debug-apk-build-GITHUB_RUN_NUMBER-GITHUB_RUN_ATTEMPT

Target commit:

GITHUB_SHA

Release notes that clearly state:

- This is an automatically generated debug APK.
- The APK was built by GitHub Actions.
- The GitHub Actions run number.
- The GitHub Actions run attempt.

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.
NO THANKS USE THE APP

- The APK is signed only with a temporary debug keystore.

Attach only the prepared .apk file as the manually uploaded release asset.

Do not manually attach:

- A ZIP file
- A source-code archive
- A tar.gz file
- The debug keystore
- Credentials
- Configuration files
- Gradle files
- Build caches
- Logs
- Secrets
- Any APK other than the verified app debug APK

GitHub may automatically display its standard Source code (zip) and Source code (tar.gz) links for the release. Do not attempt to create, upload, replace, remove, or modify those platform-generated links.

Mark the newly created release explicitly as the latest release using:

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.
NO THANKS USE THE APP

- A prerelease
- An immutable release requiring later manual publication

The APK must be visible directly under the release Assets section after successful publishing.

Release failure protection

The GitHub Actions artifact must be uploaded before release publishing begins.

Configure the release publishing step so that:

- A release error does not remove or hide the successfully uploaded Actions artifact.
- A release error does not cause the APK build result to be lost.
- A release error is shown clearly as a warning.
- The workflow summary clearly reports that the APK build and artifact upload succeeded even when release publishing fails.
- The workflow does not falsely report that release publishing succeeded.
- The release step uses continue-on-error: true.
- No unrelated fallback release action is attempted.
- No duplicate release is created as a fallback.

Assign an id to the release step so its outcome can be checked later.

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.

NO THANKS USE THE APP

This step must always run after the release attempt.

If release publishing succeeded:

- Confirm the release using gh release view.
- Print the release tag.
- Print the release title.
- Print the direct GitHub Release page URL.
- Print the exact APK asset filename.
- Add the release page URL to GITHUB_STEP_SUMMARY.
- Add a message explaining that the APK can be downloaded from the release Assets section.
- State that the normal Actions artifact is also available as a backup.

If release publishing failed:

- Print a clear warning.
- State that the APK build succeeded.
- State that the Actions artifact named app-debug-apk remains available.
- Add the same warning and backup instructions to GITHUB_STEP_SUMMARY.
- Do not falsely print a release URL.

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.

NO THANKS USE THE APP

Security restrictions

Do not create fake:

- google-services.json files
- Firebase files
- API keys
- Credentials
- Personal access tokens
- Release-signing files
- Production keystores
- Missing secrets
- Placeholder secrets

Do not expose:

- GITHUB_TOKEN
- GH_TOKEN
- Environment credentials
- Repository secrets

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.
NO THANKS USE THE APP

Do not modify repository settings.

Do not enable GitHub Pages.

Do not create a website.

Do not create a second workflow.

Do not create a release manually while preparing the repository.

The release must be created only when the generated workflow runs after I manually commit and push the workflow file.

Final validation before finishing

Before finishing, confirm all of the following:

- Only .github/workflows/build-apk.yml was created.
- No existing file was modified.
- AGP 9.1.1 was confirmed from gradle/libs.versions.toml.
- Java 17 is configured before Gradle.
- The active Java version is verified.
- Gradle 9.3.1 is used exactly.
- The repository Gradle wrapper is not used.
- The temporary debug keystore is created before the build.

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.
NO THANKS USE THE APP

- The Actions artifact name is exactly app-debug-apk.
- actions/upload-artifact is configured to fail when the APK is missing.
- The Actions artifact is uploaded before release publishing.
- The release asset is an actual .apk file and not a ZIP.
- Every workflow execution receives a unique release tag.
- GITHUB_RUN_NUMBER and GITHUB_RUN_ATTEMPT are both used in the tag.
- The release targets GITHUB_SHA.
- The release is explicitly marked as latest.
- The built-in GitHub token is used.
- The workflow has contents: write permission.
- No personal access token or repository secret is required.
- Release failure does not remove the Actions artifact.
- The workflow summary reports whether release publishing succeeded or failed.
- No Android application or project file was modified.
- No file other than .github/workflows/build-apk.yml was created.

Give me a short workflow summary that includes:

- The confirmed AGP version.

Edit with the Docs app
Make tweaks, leave comments, and share with others to edit at the same time.

NO THANKS USE THE APP

- The APK path.
- The Actions artifact name.
- The release tag format.
- The release asset filename format.
- Confirmation that the latest successful release will contain a directly downloadable APK.
- Confirmation that release failure will leave the Actions artifact available.

Do not publish, commit, sync, push, tag, or create a GitHub Release now.

Wait for me to manually commit and push the workflow file to GitHub.

Prompt End

—----------